

PROYECTO RECOMENDADOR

ÍNDICE

1. Introducción
2. Business Understanding
 - a. Objetivo General
 - b. Objetivos Específicos
3. Data Understanding
4. Data Preparation
5. Modeling
6. Infraestructura
 - a. Docker
 - b. Backend (Flask)
 - c. Frontend
 - d. Cassandra
 - e. Kafka
 - f. Zookeeper
7. Conclusión

AUTORES:

Rubén Lázaro Jiménez

Pedro Gabriel Gallardo Lozano

1. INTRODUCCIÓN

Se presenta la situación de querer mejorar una experiencia de usuario mediante la personalización de la sesión según los gustos, preferencias y costumbres para cada individuo. Este problema nos lleva a buscar generar recomendaciones personalizadas dentro de la plataforma, para hacer que el usuario obtenga una mejor experiencia, para lo cual nos hemos basado en los datos registrados de todas las personas que ya han utilizado en algún momento este servicio. Mediante el análisis de otras sesiones de usuario, podremos encontrar la mejor opción de manera personalizada para cada uno de ellos

2. BUSINESS UNDERSTANDING

2. a Objetivo General

Desarrollar un sistema de recomendación en tiempo real que entregue recomendaciones ajustadas al usuario según sus gustos y las acciones que realiza en tiempo real.

2. b Objetivos Específicos

- Búsqueda de datos para basar las recomendaciones
- EDA y limpieza de datos para poder trabajar con ellos
- Entrenamiento de modelo recomendador
- Crear un entorno con Docker en el que levantar los servicios
- Desarrollar flask con conexión a una base de datos (Cassandra)
- Desarrollar frontend para mostrar resultados de las recomendaciones
- Utilizar kafka para las conexiones en tiempo real con el usuario

3. DATA UNDERSTANDING

Para la extracción de datos hemos utilizado un dataset descargado de la web Kaggle con información de películas (título, descripción, fecha de estreno, puntuación media, cantidad de votos, productora, géneros, ruta de la imagen y si es o no para adultos) y ratings de usuarios (userId, movieId, rating del 1 al 5, timestamp) sobre esas mismas películas con formato CSV para facilitar el manejo de los mismos.

4. DATA PREPARATION

Con los datos ya obtenidos y visualizados, hemos generado un Profile Report mediante el uso de la librería yprofiling que genera un estudio en profundidad sobre cada columna dentro de los datasets, y las relaciones que existen entre las mismas. Basándonos en el documento generado, hemos analizado y limpiado los datos mediante el uso de PySpark (permitiendo trabajar con los datos en formato parquet de particiones), servicio que hemos levantado con Docker, siendo necesario para el manejo de Big Data (ya que la cantidad de datos de ratings ascendía a más de varios millones).

5. MODELING

Para realizar las predicciones y recomendaciones, lo hemos dividido en 2 procesos independientes. Uno de ellos en un modelo de filtrado colaborativo que obtiene predicciones automáticas (filtrado) sobre los intereses de cada usuario a partir de información de múltiples usuarios utilizando una matriz de factorización que se compone de usuarios, ítems y valoraciones.

En nuestro caso, hemos utilizado el modelo ALS (Alternating Least Squares) que, en nuestro caso, al tener una tabla de **id_usuario**, **id_pelicula** y **rating** correspondiente, crea una matriz de usuarios por películas y la factoriza en:

- **Matriz de factores de usuario:** una fila por usuario con vectores latentes.
- **Matriz de factores de películas:** una fila por película con vectores latentes.

Un ejemplo más visual sería el siguiente:

user_id	movie_id	rating
1	10	5
1	12	4
2	11	3
2	13	2
3	10	4
3	12	5

Teniendo la anterior matriz se procede a factorizar ambas solo con dos factores latentes teniendo el siguiente resultado:

Matriz de factores de usuario:

user_id	factor_1	factor_2
1	0.9	0.1
2	0.3	0.7
3	0.8	0.2

Matriz de factores de películas:

movie_id	factor_1	factor_2
10	1.0	0.1
11	0.2	0.9
12	0.9	0.2
13	0.5	0.5

Y el resultado final sale de **multiplicar ambas matrices**, por ejemplo, para el usuario 2 es el siguiente: $(0.3 * 0.9) + (0.7 * 0.2) = 0.27 + 0.14$.

Se ha escogido este modelo por estar integrado dentro de **pyspark**, ya que hemos tenido que tratar con millones de registros que una simple máquina no habría podido ejecutarse.

Los principales hiperparámetros de ALS son los siguientes:

- **Rank**: número de factores latentes. Mientras más alto más aprenderá el modelo pero habrá más probabilidades de **overfitting** (ajustarse demasiado a los datos de entrenamiento).
- **MaxIter**: número máximo de iteraciones del modelo. Mientras más alto más precisión pero requiere de más capacidad de cómputo.
- **RegParam**: regularización lambda para evitar sobreajuste.

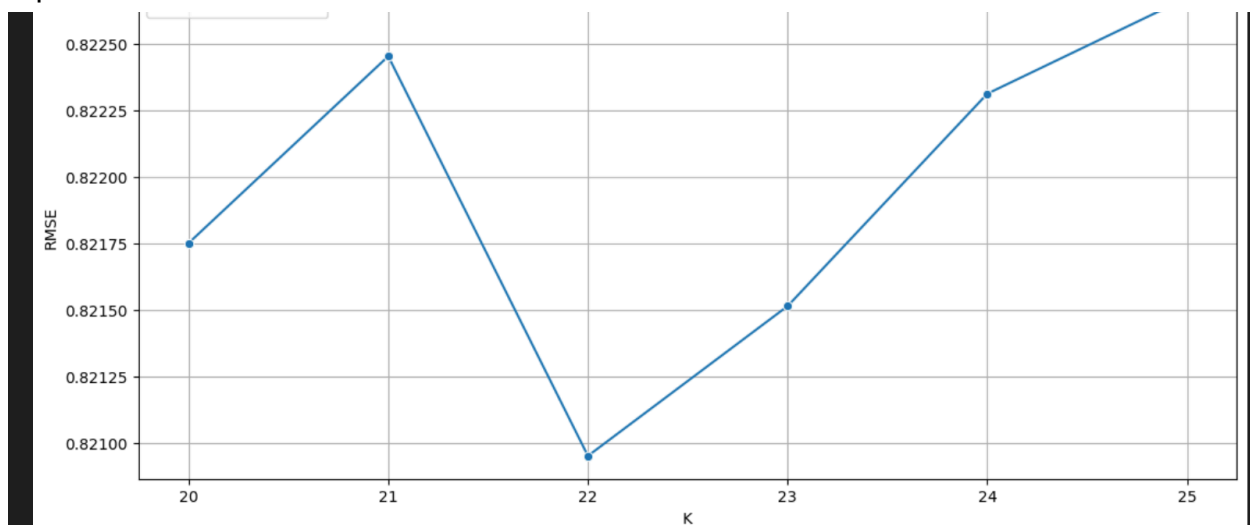
Para encontrar los mejores hiperparámetros se han probado distintas combinaciones con bucles ya que al intentar utilizar librerías como **Optuna** requería de mucha capacidad de cómputo y no era viable e incluso haciéndolo en bucles, pyspark consumía todos los recursos de RAM y de disco duro. Se optó por, primero, hacer una búsqueda **primero del mejor rank sin cambiar MaxIter ni RegParam** y luego con **dichos Ranks encontrar los mejores MaxIter y RegParam**. El resultado obtenido se observará utilizando la métrica **RMSE** (Root Mean Square Error) que penaliza los

errores grandes y el valor está en la misma unidad que los valores reales. Se muestra en la siguiente imagen:

Ranks altos para ser más precisos:

```
data = {"k":[], "rmse":[]}
(train, test) = ratings.randomSplit([0.8, 0.2], 42)
for k in range(20, 26):
    als = ALS(
        userCol="userId",
        itemCol="movieId",
        ratingCol="rating",
        nonnegative=True,
        implicitPrefs=False, # Usa False para ratings explícitos
        coldStartStrategy="drop", # Evita NaNs durante evaluación
        rank=k, # Número de "factores latentes" (equivalente a k en SVD)
        maxIter=10,
        regParam=0.1
    )
```

Aquí se ve como no varía de 0.82 de RMSE e incluso tiende a subir el error.



En la siguiente imagen se procede a utilizar unos rank más bajos teniendo como RMSE más bajo 0.8036 con **Rank: 19, MaxIter: 15 y RegParam: 0.05** que se utilizará para entrenar el modelo final. Dicho modelo se guardará en formato **pickle**:

```
rank=18, reg=0.05, iter=10 => RMSE=0.8125
rank=18, reg=0.05, iter=12 => RMSE=0.8078
rank=18, reg=0.05, iter=13 => RMSE=0.8061
rank=18, reg=0.05, iter=15 => RMSE=0.8034
rank=19, reg=0.05, iter=10 => RMSE=0.8129
rank=19, reg=0.05, iter=12 => RMSE=0.8082
rank=19, reg=0.05, iter=13 => RMSE=0.8064
rank=19, reg=0.05, iter=15 => RMSE=0.8036
rank=20, reg=0.05, iter=10 => RMSE=0.8115
rank=20, reg=0.05, iter=12 => RMSE=0.8070
rank=20, reg=0.05, iter=13 => RMSE=0.8052
rank=20, reg=0.05, iter=15 => RMSE=0.8026
rank=21, reg=0.05, iter=10 => RMSE=0.8122
rank=21, reg=0.05, iter=12 => RMSE=0.8072
rank=21, reg=0.05, iter=13 => RMSE=0.8054
rank=21, reg=0.05, iter=15 => RMSE=0.8025
```

Por otra parte también hemos utilizado un modelo de procesamiento de texto NLP, para generar recomendaciones basándonos en la similitud entre películas según su descripción o título.

Primero se han pasado a embeddings (representaciones vectoriales de texto) tanto como la descripción y el título de forma separada utilizando el modelo descargado de Hugging Face **all-MiniLM-L6-v2** y se han creado dos columnas con sus respectivos valores. Después, utilizando la distancia coseno entre los dos vectores se saca la distancia entre dichos vectores según el ángulo que formen y se devuelven los más cercanos, es decir, los más parecidos.

6. INFRAESTRUCTURA

Para el desarrollo del proyecto, hemos generado una infraestructura de trabajo pensada para poder cumplimentar de manera profesional los requisitos sobre el proyecto, priorizando la rapidez del servicio a la vez que una fácil escalabilidad.

Para el caso de **almacenamiento de datos** hemos optado por tres opciones:

- **Archivos locales:** un archivo local (.csv) que contiene la información necesaria de películas como su título, descripción, nota media, y los embeddings tanto de descripción como de título. Se ha mantenido en un archivo local debido a que los embeddings al estar contenidos en una lista cambiaran de formato y ralentizarían la ejecución del programa y también por el tamaño de dicho archivo que es demasiado grande. Para futuras actualizaciones se implementará en bases de datos SQL.
- **Base de datos SQL:** contiene la información de usuario necesaria para iniciar sesión en la aplicación. Se ha optado por utilizar supabase, que proporciona bases de datos PostgreSQL. En nuestro caso, la estructura es la siguiente:

Proyecto 4 - Proyecto Recomendador

id int8	username text	hashed_password text	created_at date	+
10	23	03ac674216f3e15c761ee1a5e255f0679536	2025-05-31	
11	2456	03ac674216f3e15c761ee1a5e255f0679536	2025-05-31	
12	123	03ac674216f3e15c761ee1a5e255f0679536	2025-05-31	
13	876	03ac674216f3e15c761ee1a5e255f0679536	2025-05-31	
14	2	03ac674216f3e15c761ee1a5e255f0679536	2025-06-07	

El usuario se registra utilizando username y contraseña, que está hasheada por seguridad.

- **Cassandra:** base de datos no relacional utilizada para guardar recomendaciones de cada usuario y eventos (acciones como dar click a botones de visto, me gusta y no me gusta) recogidos por **Kafka**. Se ha escogido esta tecnología debido a su rapidez y su similitud con bases de datos SQL en el sentido de queries y estructura de tablas. En nuestro caso existen dos tablas de Cassandra:

Tabla de recomendaciones que al iniciar sesión, en el caso de que no existan registros para dicho usuario, se llama al modelo ALS de recomendación para generar la nota y las películas (top 120 películas) y se insertan en la base de datos con esta estructura en donde **score** es la nota predicha por el algoritmo y **seen** un apartado que se utilizará cuando el usuario interactúe con el botón de marcar como visto que pasará a true para que no salga como película recomendada:

```
session.execute("""
    CREATE TABLE IF NOT EXISTS recommendations (
        user_id int,
        movie_id int,
        score float,
        titulo text,
        sinopsis text,
        img_path text,
        seen boolean,
        PRIMARY KEY (user_id, movie_id)
    ) WITH CLUSTERING ORDER BY (movie_id ASC);
""")
```

Tabla de eventos que guarda información de los eventos lanzados por cada usuario al interactuar con la página en donde entra en juego **Kafka**. Cuando se inicializa la aplicación se crea un producer que manda información (id_usuario, id_pelicula, acción y hora del evento) al topic **eventos_pelicula** en el cual hay un consumer que está escuchando y esperando a la llegada de información, que

en nuestro caso al llegar la información a dicho consumer, se guardará en la tabla de eventos de Cassandra. La estructura es la siguiente:

```
session.execute("""
    CREATE TABLE IF NOT EXISTS eventos_usuario (
        user_id int,
        accion text,
        movie_id int,
        timestamp timestamp,
        PRIMARY KEY ((user_id, accion), movie_id, timestamp)
    ) WITH CLUSTERING ORDER BY (movie_id ASC, timestamp DESC);
""")
```

Se observa que en Primary Key hay una clave primaria compuesta a la vez por dos campos (`user_id` y `acción`) ya que se necesitaba filtrar por ambos campos. Al haber guardado la información en dicha tabla, el usuario al recargar la página o volver a iniciar sesión verá que las películas que ha marcado como vistas han pasado a otro apartado y ya no saldrán en las recomendaciones iniciales. Esto lo hacemos obteniendo los eventos del usuario que sean de **visto** (utilizando las claves primarias previamente comentadas) y filtramos la tabla de recomendaciones en busca de dicha película para marcar el campo **seen** como true.

6. a Docker

Todos los servicios se han decidido levantarlos en Docker, un sistema de contenedores que favorece la escalabilidad y la compatibilidad entre distintas máquinas. En nuestro caso se ha optado por la creación de **5 contenedores** (backend, frontend, cassandra, Kafka y zookeeper).

6. b Backend (Flask)

El servicio de acción backend hemos utilizado Flask, gracias al cual podemos conectar mediante librerías con el frontend y la base de datos, así como la utilización de modelos para generar recomendaciones y mandarlas fácilmente mediante formato json.

6. c Frontend

Se han utilizado tecnologías como **React y typescript** para la estructura de la parte de cliente basada en plantillas que muestran el diseño web junto con la información recibida mediante jsons del backend.

Mencionar que el contenedor depende del backend ya que el backend necesita de tiempo para ejecutarse y se ha optado por la creación de un shell que nos garantiza que hasta que no se ha lanzado completamente el backend, el frontend no se lanzará. Se ha optado por la anterior opción para que el frontend no se quede pillado intentando cargar información.

6. d Cassandra

Este contenedor no tiene utilidad ya que las conexiones y la ingesta de datos no se realizan al contenedor (como se pensó inicialmente). Se ha mantenido para no dañar la estructura ya previamente montada de contenedores y dependencias

6. e Kafka

Contenedor que proporciona la infraestructura necesaria para la mensajería de lotes de información que es producida por el usuario (**producer**) al interactuar con la página (hacer clicks en botones como marcar como visto, me gusta o no me gusta), enviada al **topic** eventos_pelicula y escuchada por el **consumer** (**puerto kafka:9092**) que se encuentra dentro de un **consumer group** (cassandra-consumer-group) que envía la información a las tablas pertinentes de Cassandra.

6. f Zookeeper

Este contenedor actúa como servicio para guardar información sobre los brokers de Kafka, topics y particiones.

7. CONCLUSIÓN

Como resultado propuesto al problema emergente, cumpliendo con los requisitos determinados para la resolución del mismo, hemos desarrollado una plataforma con acceso mediante usuarios (diseño de login y registro de usuario), un repositorio bastante amplio de películas con toda la información y categorización que se necesita, incluyendo ratings o puntualizaciones sobre las mismas.

Esta plataforma cuenta con la capacidad de interacción en tiempo real con el usuario para definir acciones sobre las películas.

También cuenta con un sistema de recomendación de búsqueda para encontrar películas con características similares, así como un sistema de recomendaciones personalizado que se adecua al perfil de cada cliente, para obtener una mejor experiencia y personalizada para cada usuario.